

Module - II

Stack, Queue & Linked List

Linked List

A linked list is a linear data structure in which elements are stored in separate memory locations and connected using pointers (links). Unlike an array, the elements of a linked list are not stored in contiguous memory locations.

Each element of a linked list is called a node and each node contains two parts:

1. Data - stores the actual value.
2. Pointer (Next) - stores the address of the next node, if last node (i.e. tail node) then stores NULL.

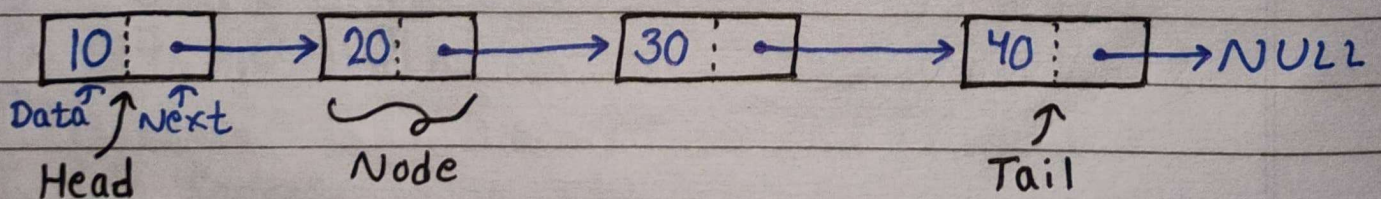


Fig.: Simple Linked List.

Creating a Node in C:

In C, a node is created using structure, defined using the "struct" keyword. It is a user-defined data type that allows us to group related variables of different data types together under a single name.

Defining

```
struct Node
```

```
{  
    int data;  
    struct Node *next;  
};
```

Creating

```
struct Node *newNode;  
newNode = malloc  
    (sizeof(struct Node));  
newNode → data = 10;  
newNode → next = NULL;
```

OR

```
typedef struct
```

```
{  
    int data;  
    struct Node *next;  
} Node;
```

```
Node *newNode;
```

Types of Linked List:

1. Singly Linked List
2. Doubly or Two-way Linked List
3. Circular Linked List
4. Doubly Circular Linked List.

Operations on Linked List:

1. Traversal
2. Search
3. Insertion
4. Deletion

Singly Linked List Implementation in C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node  
{
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
int main()  
{
```

```
    struct Node *head, *second, *third, *tail;
```

```
    head = malloc(sizeof(struct Node));
```

```
    second = malloc(sizeof(struct Node));
```

```
    third = malloc(sizeof(struct Node));
```

```
    tail = malloc(sizeof(struct Node));
```

```
    head->data = 10;
```

```
    head->next = second;
```

```
second → data = 20;  
second → next = third;
```

```
third → data = 30;  
third → next = tail;
```

```
tail → data = 40;  
tail → next = NULL;
```

```
printf("%d %d %d %d" head → data,  
second → data, third → data, tail → data);
```

```
free(head);  
free(second);  
free(third);  
free(tail);
```

```
return 0;
```

```
}
```

Output:

10 20 30 40

Operations on Singly Linked List:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

[For Exam]

```
int main() // Traversal
```

```
void display(struct Node *head)
```

```
{
```

```
    // temp is used not directly head. struct Node *temp;
```

```
    if (head == NULL)
```

```
    {
```

```
        printf("List is empty \n");
```

```
        return;
```

```
    }
```

```
    printf("Linked List: ");
```

```
    // temp = head;
```

```
    while (head != NULL)
```

```
    {
```

```
        printf("%d → ", temphead->data);
```

```
        temphead = temphead->next;
```

```
    }
```

```
    printf("NULL \n");
```

```
}
```

[For Exam] // Insertion (Beginning, End & Position)

```
void insertAtBeginning(struct Node **head,  
                        int new_data)
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = malloc(sizeof(struct Node));
```

```
    newNode → data = new_data;
```

```
    newNode → next = *head;
```

```
    *head = newNode;
```

```
}    printf("Inserted %d at the beginning.\n", new_data);
```

```
void insertAtEnd(struct Node **head,  
                 int new_data)
```

```
{
```

```
    struct Node *newNode, *temp;
```

```
    newNode = malloc(sizeof(struct Node));
```

```
    newNode → data = new_data;
```

```
    newNode → next = NULL;
```

```
    if (*head == NULL)  
    {
```

```
        *head = newNode;
```

```
        printf("Inserted %d as the  
                head node.\n", new_data);
```

```
        return;
```

```
    }
```

```
temp = *head;
```

```
while (temp → next != NULL)  
{
```

```
    temp = temp → next;  
}
```

```
temp → next = newNode;
```

```
printf("Inserted %d at the end.\n",  
      new_data);
```

```
}
```

```
void insertAtPosition (struct Node **head,  
                      int new_data, int pos)
```

```
{
```

```
    if (pos < 1)  
    {
```

```
        printf("Invalid position.\n");  
        return;
```

```
    }
```

```
    if (pos == 1)  
    {
```

```
        insertAtBeginning(head, new_data);  
        return;
```

```
    }
```

```
    struct Node *new_node, *temp;
```

```
    temp = *head;
```

```
    new_node = malloc(sizeof(struct Node));
```

```

for (int i = 1; temp != NULL && i < pos - 1;
     i++)
{
    temp = temp → next;
}

if (temp == NULL)
{
    printf("Position not available");
    return;
}

```

```

new_node → data = new_data;
new_node → next = temp → next;
temp → next = new_node;

```

```

printf("Inserted %d at position %d.\n",
       new_data, pos);

```

[For Exam]

```

// Deletion (Beginning, End & Position)
void deleteFromBeginning(struct Node **head)
{
    if (*head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    struct Node *temp = *head;
    *head = (*head) → next;
    printf("Deleted %d from the beginning.\n",
           temp → data);
    free(temp);
}

```

```
int main()
{
    struct Node *head = NULL;

    insertAtBeginning(&head, 20);
    insertAtEnd(&head, 30);
    insertAtPosition(&head, 10, 1);
    insertAtPosition(&head, 25, 3);

    deleteFromBeginning(&head);

    display(head);

    return 0;
}
```

Output:

Inserted 20 at the beginning.

Inserted 30 at the end.

Inserted 10 at the beginning.

Inserted 25 at position 3.

Deleted 10 from the beginning.

Linked List: 20 → 25 → 30 → NULL

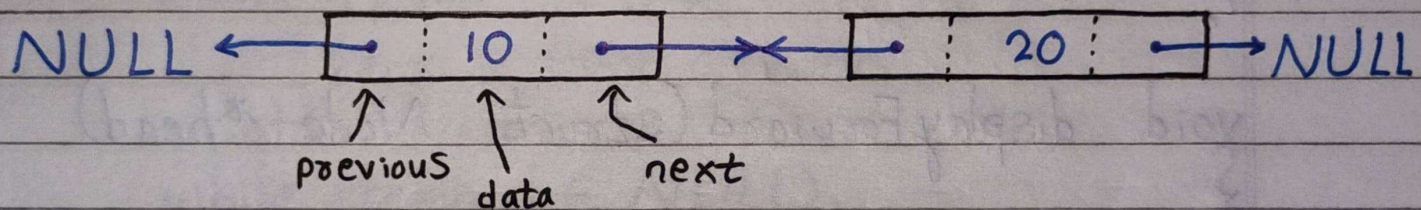
Doubly / Two-way Linked List

A doubly or two-way linked list is a linked list in which each node contains three parts:

prev → pointer to the previous node.

data → stores the data.

next → pointer to the next node.



Basic Operations:

1. Insertion
2. Deletion
3. Traversal (Forward & Backward).

Note: Doubly linked list is useful when two-way traversal is required. It requires extra memory for the prev (previous) pointer.

Program:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *prev;
```

```
    struct Node *next;
```

```
};
```

```
void displayForward(struct Node *head)
```

```
{
```

```
    struct Node *temp = head;
```

```
    if (temp == NULL)
```

```
    {
```

```
        printf("List is empty.\n");
```

```
        return;
```

```
    }
```

```
    printf("NULL → ");
```

```
    while (temp != NULL)
```

```
    {
```

```
        printf("%d → ", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("NULL");
```

```

void displayBackward (struct Node *head)
{
    struct Node *temp = head;

    if (temp == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    while (temp → next != NULL)
        temp = temp → next;
    printf("NULL ← ");
    while (temp != NULL)
    {
        printf("%d ← ", temp → data);
        temp = temp → next prev;
    }
    printf("NULL");
}

```

```

int main()
{

```

```

    struct Node *first, *second;

```

```

first = malloc(sizeof Node)

```

```

    first = malloc(sizeof(struct Node));

```

```

    second = malloc(sizeof(struct Node));

```

```

    first → data = 10;

```

```

    first → prev = NULL;

```

first $\rightarrow$ next = second;

second $\rightarrow$ data = 20;

second $\rightarrow$ prev = first;

second $\rightarrow$ next = NULL;

printf("Forward: ");

displayForward(first);

printf("\n\nBackward: ");

displayBackward(first);

free(first); free(second);

return 0;

}

Output:

Forward: NULL $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ NULL

Backward: NULL $\leftarrow$ 20 $\leftarrow$ 10 $\leftarrow$ NULL

Header Linked List

A Header Linked List is a ^{special} type of linked list that contains a special header node at the beginning of the list.

The header node is used to store information about the list or to simplify list operations. It normally does not store ordinary data.

Advantages:

- Makes insertion and deletion at the beginning easier.
- Can store additional information about the list.
- Helps handle special cases such as an empty list.

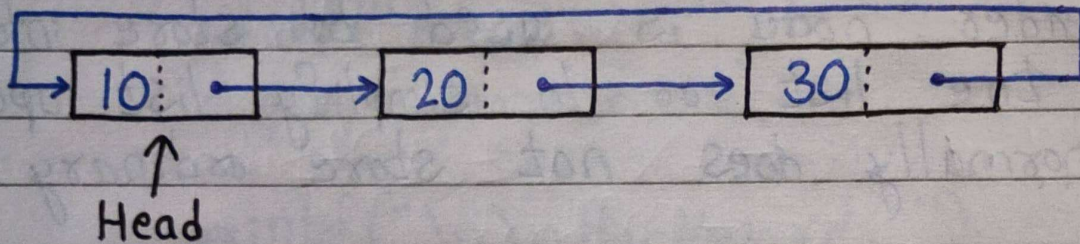
Types:

- Singly Header Linked List
- Doubly / Two-way Header Linked List
- Circular Header Linked List
- Circular Two-way Header Linked List

Note: A two-way or doubly header linked list combines Doubly Linked List and Header Linked List.

Circular Linked List

A Circular Linked List is a linked list in which the last node points back to the first node instead of pointing to NULL.



Note: A condition such as $temp \neq head$ is used to stop traversal and do while loop is used.

Advantages:

- Useful for application that require repeated or cyclic processing.
- Traversal can continue from the last node directly to the first.
- Used in circular queues.

Types:

- Circular Singly Linked List
- Circular Doubly Linked List.

Array of Linked List

An array of linked list is an array in which each element stores the address of the first node of a linked list.

<u>Array</u>	
0	$\Rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$
1	$\Rightarrow 40 \rightarrow 50 \rightarrow \text{NULL}$
2	$\Rightarrow \text{NULL}$

Simply it means, each array position can represent a separate linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node
{
```

```
    int data;
    struct Node *next;
```

```
};
```

```
int main()
{
```

```
    struct Node *head[2];
```

```
    head[0] = malloc(sizeof(struct Node));
    head[1] = malloc(sizeof(struct Node));
```

```
head[0] → data = 10;  
head[0] → next = malloc(sizeof(struct Node));  
head[0] → next → data = 20;  
head[0] → next → next = NULL;
```

```
head[1] → data = 30;  
head[1] → next = NULL;
```

```
printf("List 1: %d → %d \n",  
      head[0] → data,  
      head[0] → next → data);
```

```
printf("\n List 2: %d \n", head[1] → data);
```

```
free(head[0] → next);  
free(head[0]);  
free(head[1]);
```

```
return 0;
```

```
}
```

Output:

List 1: 10 → 20

List 2: 30

Linked List Using Array [Static Linked List or Cursor Implementation.]

A linked list can be created using only an array instead of pointers. In this, the next field stores the index of the next node in the array.

Representation:

Index	Data	Next
0	10	1
1	20	2
2	30	-1 (NULL)

Program:

```
#include <stdio.h>
```

```
struct Node
```

```
{  
    int data;
```

```
    int next;
```

```
};
```

```
int main()
```

```
{
```

```
    struct Node list[3];
```

```
    list[0].data = 10;
```

```
    list[0].next = 1;
```

```
list[1].data = 20;  
list[1].next = 2;
```

```
list[2].data = 30;  
list[2].next = -1;
```

```
printf("Linked List: ");
```

```
int i = 0;
```

```
while (i != -1)
```

```
{
```

```
    printf("%d → ", list[i].data);  
    i = list[i].next;
```

```
}
```

```
printf("NULL");
```

```
return 0;
```

```
}
```

Output:

Linked List: 10 → 20 → 30 → NULL

Note: In this method, the size of the linked list is limited by the size of the array.

Generalized Linked List (GLL)

A Generalized Linked List (GLL) is a linked list in which an element can be either an ordinary data element or a pointer to another generalized linked list. This means, a GLL can contain another list as an element.

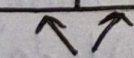
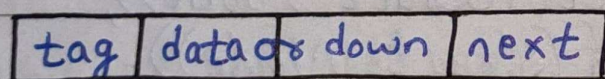
Each node in GLL contains:

tag $\rightarrow$ 0 (for no sub-list) or 1 (sub-list)
(or flag)

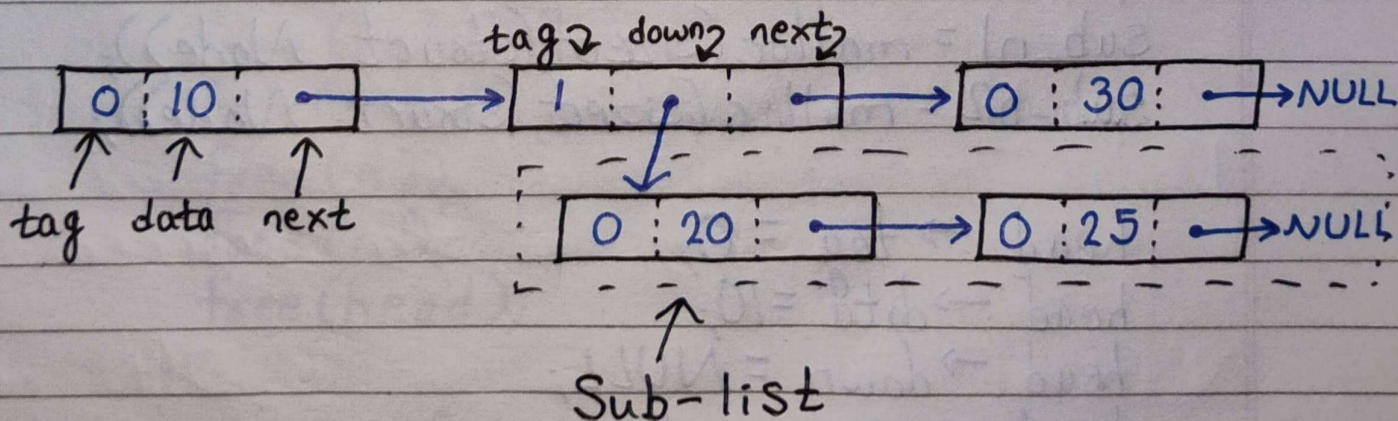
data $\rightarrow$ The actual atomic data [tag == 0]

down $\rightarrow$ Stores the address of sub-list first node.
[tag == 1]

next $\rightarrow$ Stores the address of next node in main list.



Based on tag (i.e. Sub-list or not)



Note: Tag 0 (use data), Tag 1 (use down).

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    int tag;
```

```
    int data;
```

```
    struct Node *down;
```

```
    struct Node *next;
```

```
};
```

```
void display(struct Node *head);
```

```
int main()
```

```
{
```

```
    struct Node *head, *n1, *n2, *sub_n1, *sub_n2;
```

```
    head = malloc(sizeof(struct Node));
```

```
    n1 = malloc(sizeof(struct Node));
```

```
    n2 = malloc(sizeof(struct Node));
```

```
    sub_n1 = malloc(sizeof(struct Node));
```

```
    sub_n2 = malloc(sizeof(struct Node));
```

```
    head → tag = 0;
```

```
    head → data = 10;
```

```
    head → down = NULL;
```

```
    head → next = n1;
```

```
n1 → tag = 1;  
n1 → data = 0; // Not Used  
n1 → down = sub_n1;  
n1 → next = n2;
```

```
sub_n1 → tag = 0;  
sub_n1 → data = 20;  
sub_n1 → down = NULL;  
sub_n1 → next = sub_n2;
```

```
sub_n2 → tag = 0;  
sub_n2 → data = 25;  
sub_n2 → down = NULL;  
sub_n2 → next = NULL;
```

```
n2 → tag = 0;  
n2 → data = 30;  
n2 → down = NULL;  
n2 → next = NULL;
```

```
printf("G L L \n\n");  
display(head);
```

```
free(sub2);  
free(sub1);  
free(n2);  
free(n1);  
free(head);
```

```
return 0;
```

```

void display (struct Node *head)
{
    struct Node *temp = head;
    while (temp != NULL)
    {
        if (temp -> tag == 0)
        {
            printf("%d ", temp -> data);
        }
        else
        {
            printf("(");
            display(temp -> down);
            printf(")");
        }
        temp = temp -> next;
        if (temp != NULL)
            printf(" -> ");
    }
    printf(" -> NULL");
}

```

Output:

GLL

10 $\rightarrow$ (20 $\rightarrow$ 25 $\rightarrow$ NULL) $\rightarrow$ 30 $\rightarrow$ NULL

Garbage Collection

Garbage Collection is the process of automatically identifying and freeing the memory occupied by data or nodes that are no longer being used by a program.

In C, there is no any built-in garbage collection. Instead, C relies on manual memory management. Normally `free()` is used.

Compaction

Compaction is a memory management technique used to reduce fragmentation by moving allocated memory blocks together and creating one large continuous free space.

When dynamic memory is allocated and released, free spaces may become scattered. Although there may be enough total free memory, it may not be available as one continuous block. Thus, compaction is needed.

Example:

Used | Free | Used | Free | Used | Used | Free

↓ After Compaction

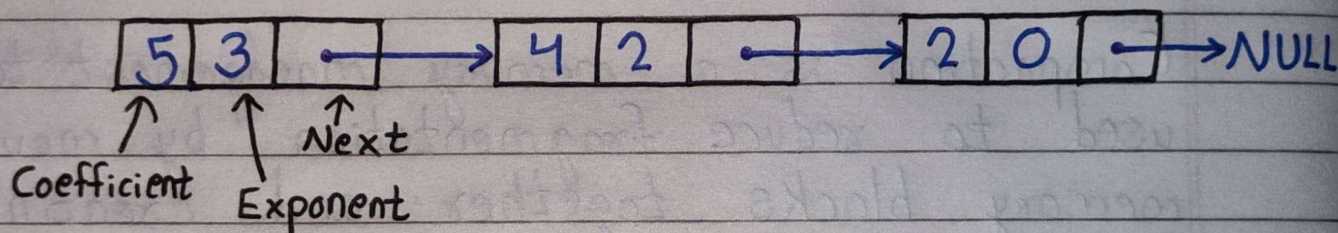
Used | Used | Used | Used | Free. Free. Free

Polynomial Arithmetic Using Linked List

Polynomial Arithmetic means performing operations such as addition, subtraction and multiplication on polynomials (such as $5x^3 + 4x^2 + 2$).

A polynomial can be represented using a linked list where each node stores one term of the polynomial.

For the polynomial, $5x^3 + 4x^2 + 2$, each node contains Coefficient, Exponent & Next.



For addition, $(5x^3 + 3x^2 + 2) + (4x^3 + 5x^2 + 1)$
 $= 9x^3 + 8x^2 + 3$.